CITY
ST GEORGE'S
UNIVERSITY OF LONDON

Computer Graphics Using OpenGL
INM376 / IN3005

Module Leader – Eddie Edwards

Lecture 04: Shaders and colour

Recap from last time

- Viewing
  - Modeling
  - Viewing
  - Projection
  - Viewport
- VBOs and VAOs
  - Retained mode rendering
  - Generic storage of vertex attributes
- Catmull-Rom splines
  - Cubic splines that interpolate the control points
  - Are smooth (continuous with continuous derivatives)

Ed Catmull, president of Walt Disney Animation Studios and Pixar Animation Studios

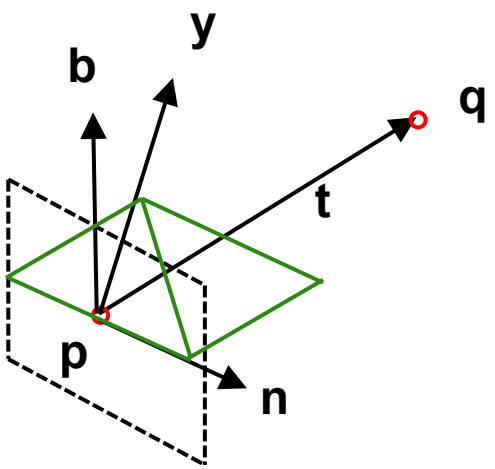
Lab 3 solution

Part 4: Catmull-Rom spline interpolation

```
glm::vec3 CCatmullRom::Interpolate(const glm::vec3 &p0, const glm::vec3 &p1, const glm::vec3 &p2,
const glm::vec3 &p3,float t) {
   float t2 = t * t;
   float t3 = t2 * t;
   glm::vec3 a = p1;
   glm::vec3 b = 0.5f * (-p0 + p2);
   glm::vec3 c = 0.5f * (2.0f*p0 - 5.0f*p1 + 4.0f*p2 - p3);
   glm::vec3 d = 0.5f * (-p0 + 3.0f*p1 - 3.0f*p2 + p3);
   glm::vec3 x = a + b*t + c*t2 + d*t3;
   return x;
}


void Game::Update() {
   glm::vec3 p0 = glm::vec3(-500, 10, -200); glm::vec3 p1 = glm::vec3(0, 10, -200);
   glm::vec3 p2 = glm::vec3(0, 10, 200); glm::vec3 p3 = glm::vec3(-500, 10, 200);

   static float t = 0.0f;
   t += 0.0005f * (float) m_dt;
   if (t > 1.0f)
      t = 0.0f;

   CCatmullRom cr;
   glm::vec3 p = cr.Interpolate(p0, p1, p2, p3, t);
   m_pCamera->Set(p, glm::vec3(0, 0, 0), glm::vec3(0, 1, 0));
}
```

Tangents and the NB plane

- Given a Catmull-Rom spline, it's easy to find a vector tangent to the curve. It is also possible to parameterise the plane orthogonal to the tangent vector.



t = q - p
n = t x y
b = n x t

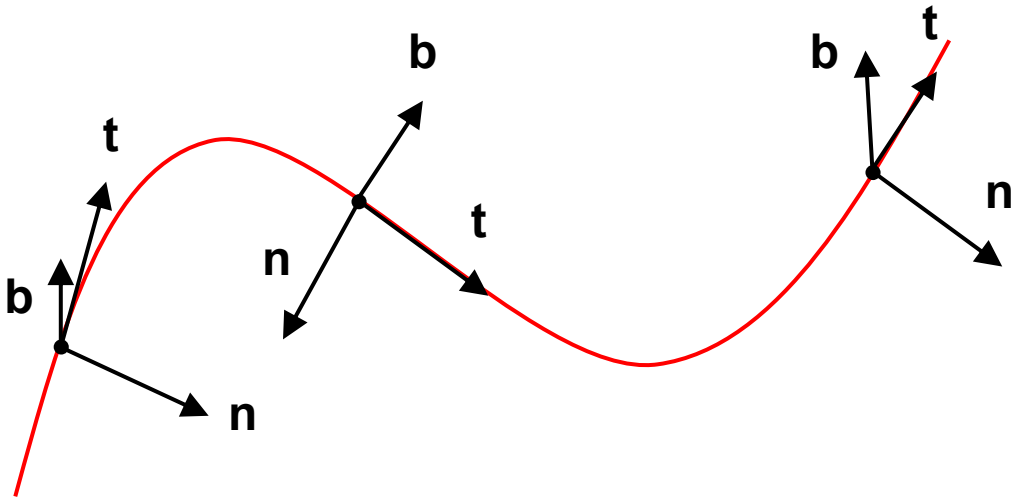
frame

normalise **t**, **n** and **b**
create track geometry in the **tn** plane

n and **b** are called the *normal* and *binormal*

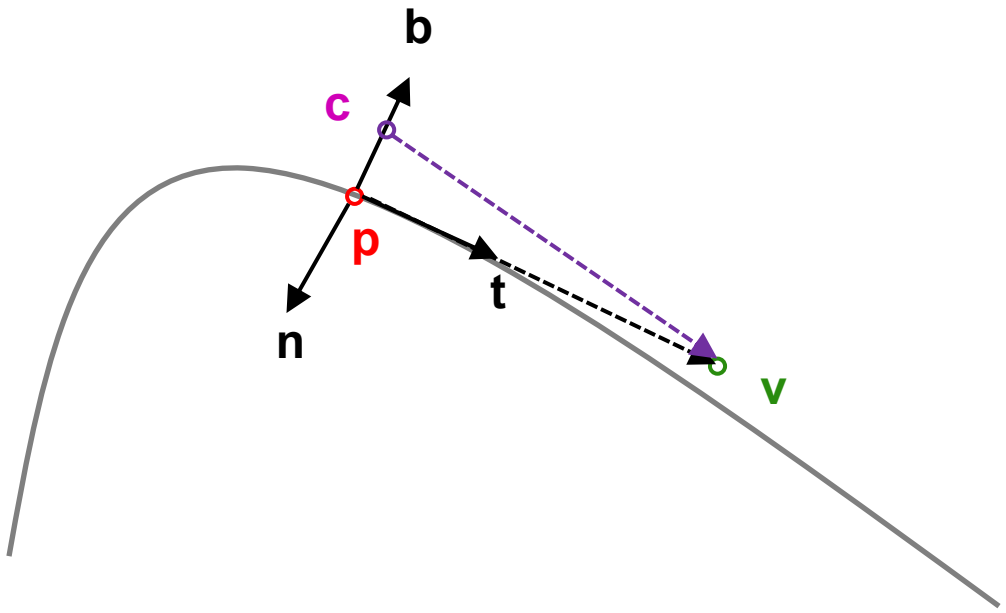
TNB frame

- A frame is a local coordinate system that may change along a space curve
- The **t**, **n**, and **b** vectors are mutually orthogonal, kind of like the **x**, **y**, and **z** axes. However, unlike the **xyz** axes, the **tnb** frame changes along the space curve.
- The **n** and **b** vectors specify the plane orthogonal to **t**
- A TNB frame is also known as a *Frenet frame or Frenet Serret Frame*
- When **t**, **n**, **b** are normalised and mutually orthogonal, they are said to form an *orthonormal basis* spanning 3D space



The camera and the TNB frame

- If one has a TNB frame (with associated normalised vectors), one can set the camera position **c** to be at a height h from a point **p** along the **b** vector. The viewpoint **v** can be at a point along the tangent vector, a length l units from **p**. Other possibilities exist (such as moving the camera down along the **b** vector and/or moving it back along the negative **t** vector, etc.)



$\mathbf{c} = \mathbf{p} + h\mathbf{b}$

$\mathbf{v} = \mathbf{p} + l\mathbf{t}$

Overview of this lecture

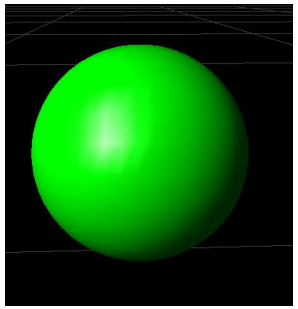
- Shaders
  - History
  - Creating and using shaders
  - The OpenGL Shading Language (GLSL)
  - Graphics client / server communication
  - Examples of vertex and fragment shaders
- Colour
  - RGB
  - RGBA

Limitations of the fixed function pipeline

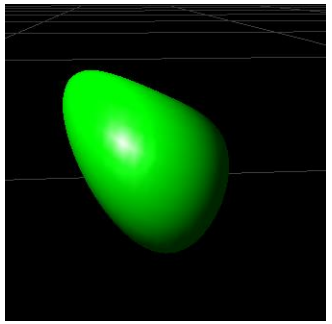
- Although the fixed function pipeline contains many functions that affect the rendering, the programmer is limited to what can be achieved using these functions.
- For example, for some programmers, it may be too limiting to require
  - Linear transformations of vertices
  - Interpolated colour
  - Only eight lights
  - The Blinn-Phong reflectance model
  - Per-vertex lighting
  - (more)

What is a shader?

- A shader is a **server-side computer program** that is used primarily to produce rendering effects on graphics hardware.
- As the name implies, shaders can implement lighting and shading effects. But they are capable of doing *much more*.
- A shader provides an alternative to, and is more flexible than, the fixed function pipeline.
- Through programming, it is possible to create effects with shaders that are *difficult or impossible* to produce with the fixed function pipeline.
- A few examples:
  - Non-linear spatial transformations
  - Non-photorealistic rendering
  - Per-fragment lighting



Sphere



Nonlinear transformation

Shader gallery

ATI demo
https://www.youtube.com/watch?v=bFhvS8WA9eM&ab_channel=Bernie2018

Boulanger PhD thesis
https://www.youtube.com/watch?v=NAicuxAJ8Wo&t=19s&ab_channel=WillBurns

Orkun Bektas MSc thesis
https://www.youtube.com/watch?v=pIN8EbLvdmU&ab_channel=OrkunBektas

A brief history of shaders

- The term “shader” was coined by Pixar, with their RenderMan software. Initially shaders were used for offline rendering, e.g., Toy Story.
- Late 1990s: PixelFlow and RealTime Shading language offered software support for real-time shaders.
- Consumer-grade graphics cards were introduced to the marketplace in the mid 1990s; however, they did not support shaders until early 2000s.
- In 2004, OpenGL 2.0 was released, including GLSL, a C-style shading language.
- These days, dedicated graphics cards will have hardware support for shaders. Integrated chipsets may support shaders through software if hardware support is not available.
- Programmable pipelines are now standard in modern computer graphics.

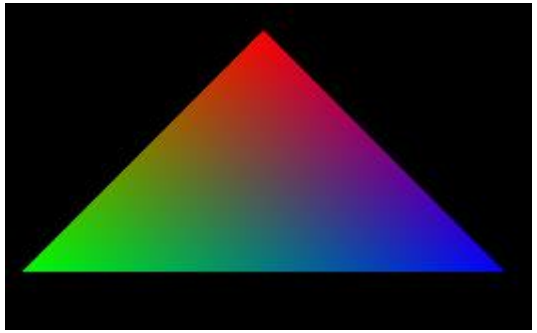
NVidia GTX 970

Vertex shaders

- Vertex shaders do calculations on **vertices**, such as
  - Transform a vertex position
  - Evaluate lighting at a vertex
  - Calculate vertex colour
- Vertex shaders typically take *vertex attributes* (from a VBO) as input data
  - Vertex attributes include position, colour, texture coordinates, normal
  - Vertex shaders typically have knowledge of the matrices (e.g., modelview matrix, projection matrix) used to transform points.
- Vertex shaders operate after a call to `glDrawArrays` or `glDrawElements`.
- The vertex shader runs once for each vertex.

Fragment shaders

- Fragment shaders compute the colour of a **fragment**. They may draw upon different attributes at the fragment, including
  - Colour
  - Surface normal
  - Depth (distance from camera)
  - Texture coordinate and texture
  - (more)
- A simple fragment shader might just interpolate between the colour values passed in from vertex shaders or it can introduce complex processing.
- The fragment shader runs once for each fragment.



Cores and parallelisation

- The OpenGL pipeline has been designed to be *data parallel*. For example, processing a fragment can be done independently of other fragments.
- Therefore, a shader program can be run in parallel for each fragment, running simultaneously on different threads.
- Modern GPUs have a massive number of cores. For example, the NVidia GeForce GTX 970 has *1664 CUDA cores*. Each core can do vertex, geometry, or fragment shader calculations.

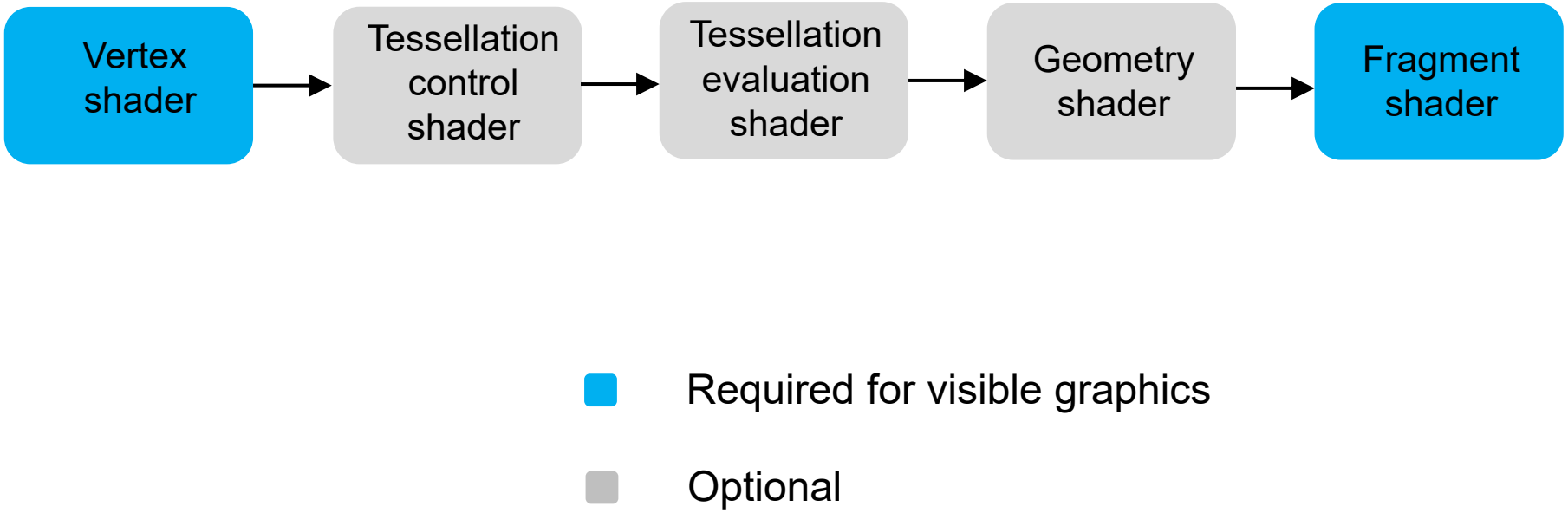
⇨ Although shaders were originally designed for graphics, their parallel nature combined with performance of GPUs have found new applications beyond graphics. General purpose GPU (GP-GPU) programming is an interesting topic; different programming mechanisms exist (compute shaders, OpenCL, etc.).

Shader languages

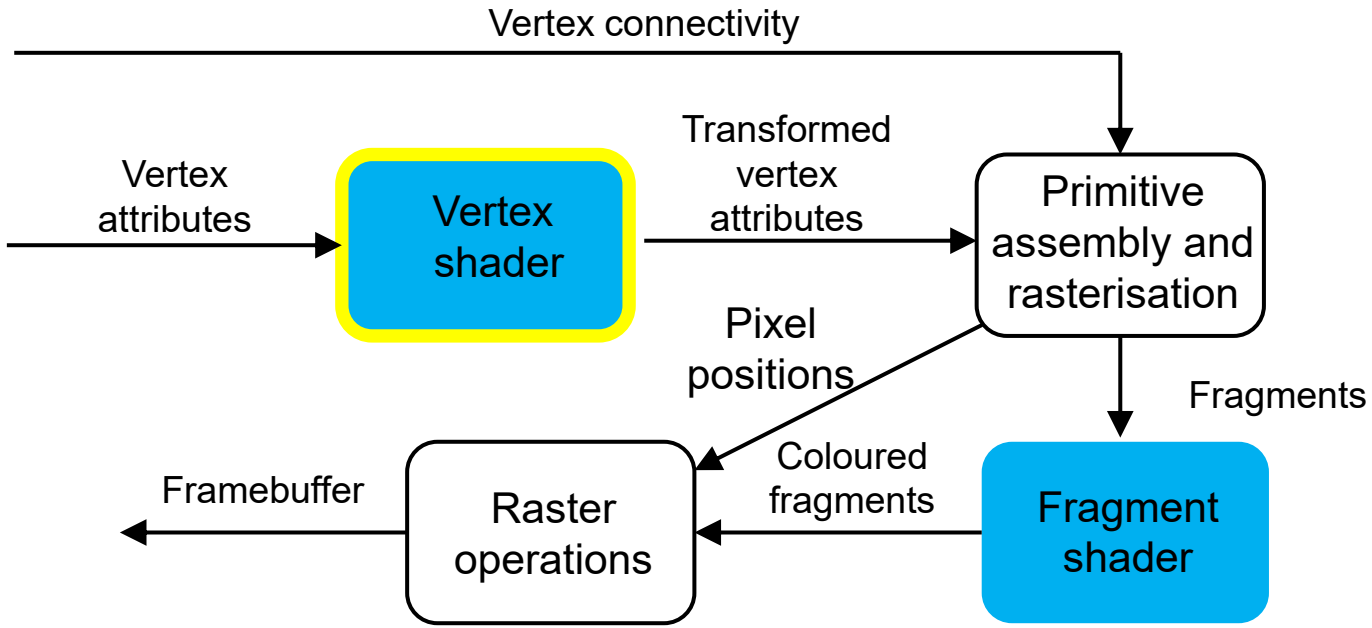
- Originally, shaders were written in assembly language. However, this is complex and unwieldy. High level shader languages provide a simpler, more familiar way to program shaders.
- OpenGL shading language (**GLSL**), sometimes pronounced “GL-slang,” offers a C-like language that is designed to work with OpenGL.
- High level shading language (**HLSL**) is a proprietary shading language developed by Microsoft for use with Direct3D.

OpenGL 4.0 shader pipeline

- Simplified view

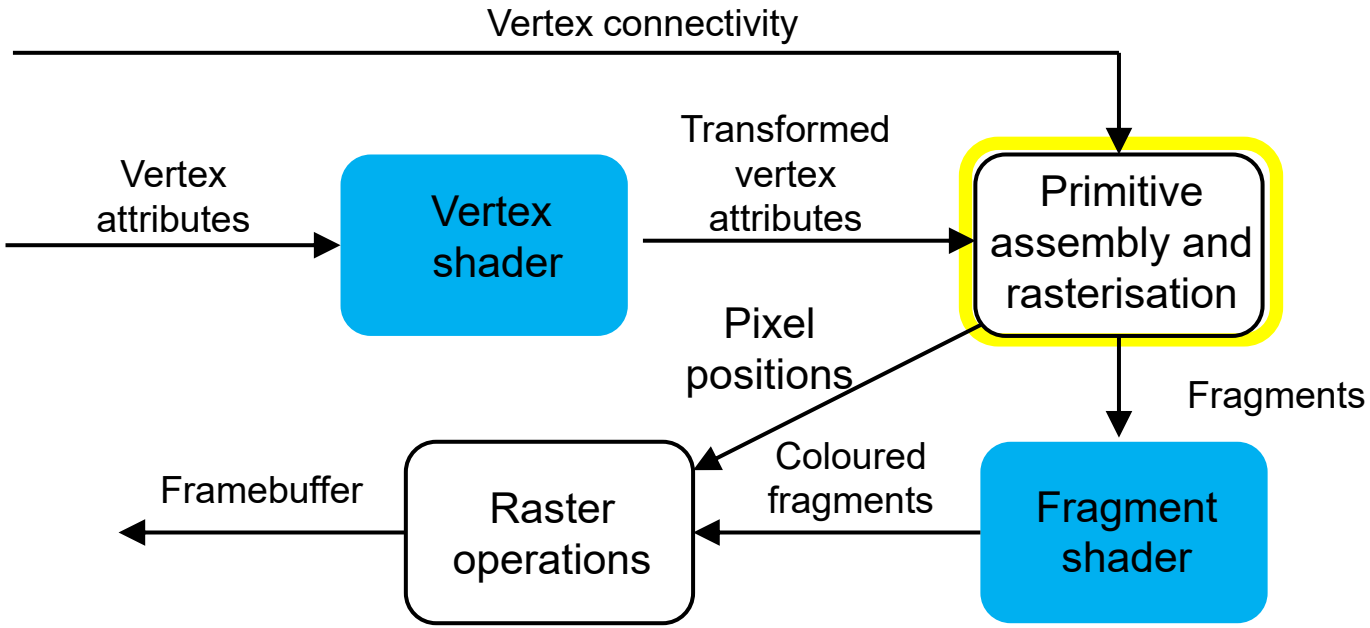


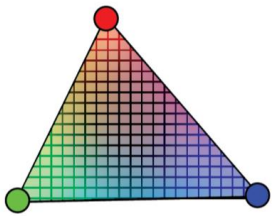
Detailed view: vertex / fragment shaders



- *Vertex shader*
  - Inputs: individual vertex attributes (position, colour, normal, texture coord)
  - Typical operations: position transformation, lighting computations per vertex, generation and transformation of texture coordinates
  - Outputs: transformed vertex attributes.

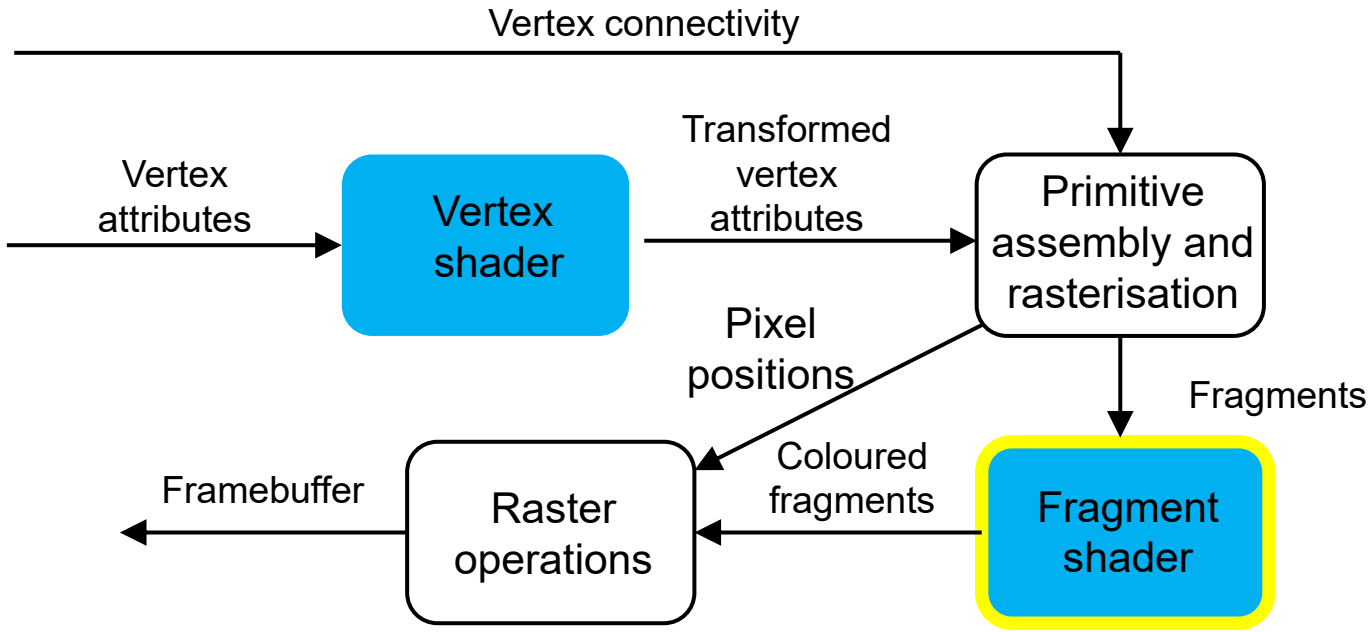
Detailed view: vertex / fragment shaders





- *Primitive assembly and rasterisation*
  - Inputs: transformed vertices and connectivity information
  - Typical operations: determining fragment locations, clipping against the view frustum, back face culling
  - Outputs: fragment positions, fragments to be coloured

Detailed view: vertex / fragment shaders

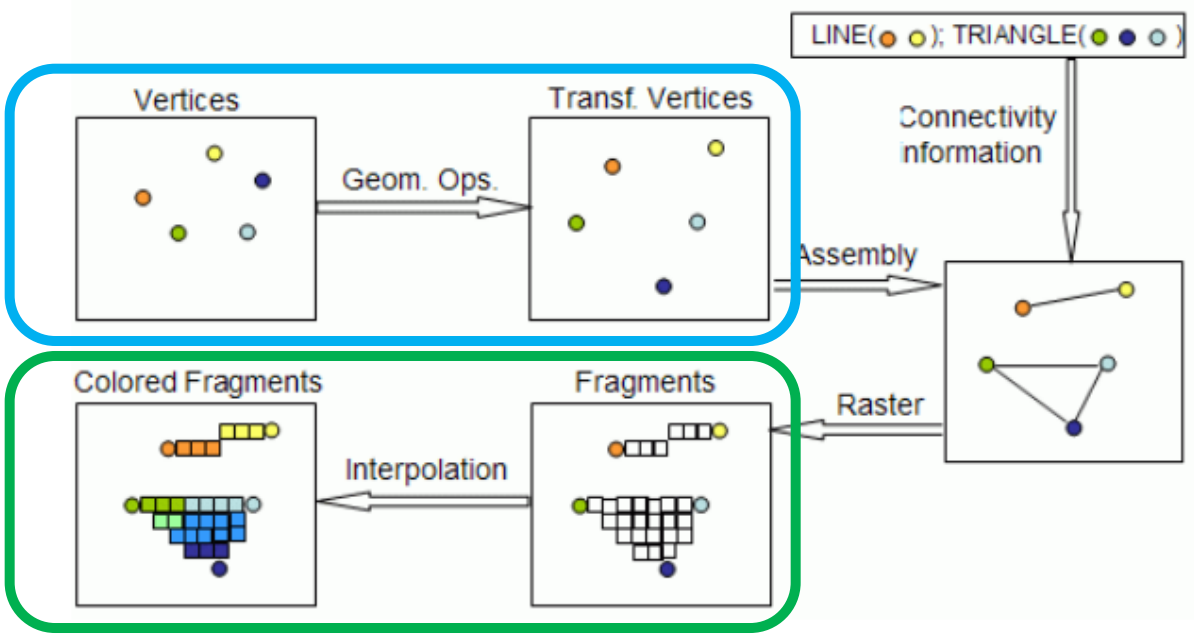


- *Fragment shader*
  - Input: fragments
  - Typical operations: assigning a fragment colour based on interpolated colour, texel, etc.
  - Output: coloured fragments

Example

Vertex shader

Fragment shader

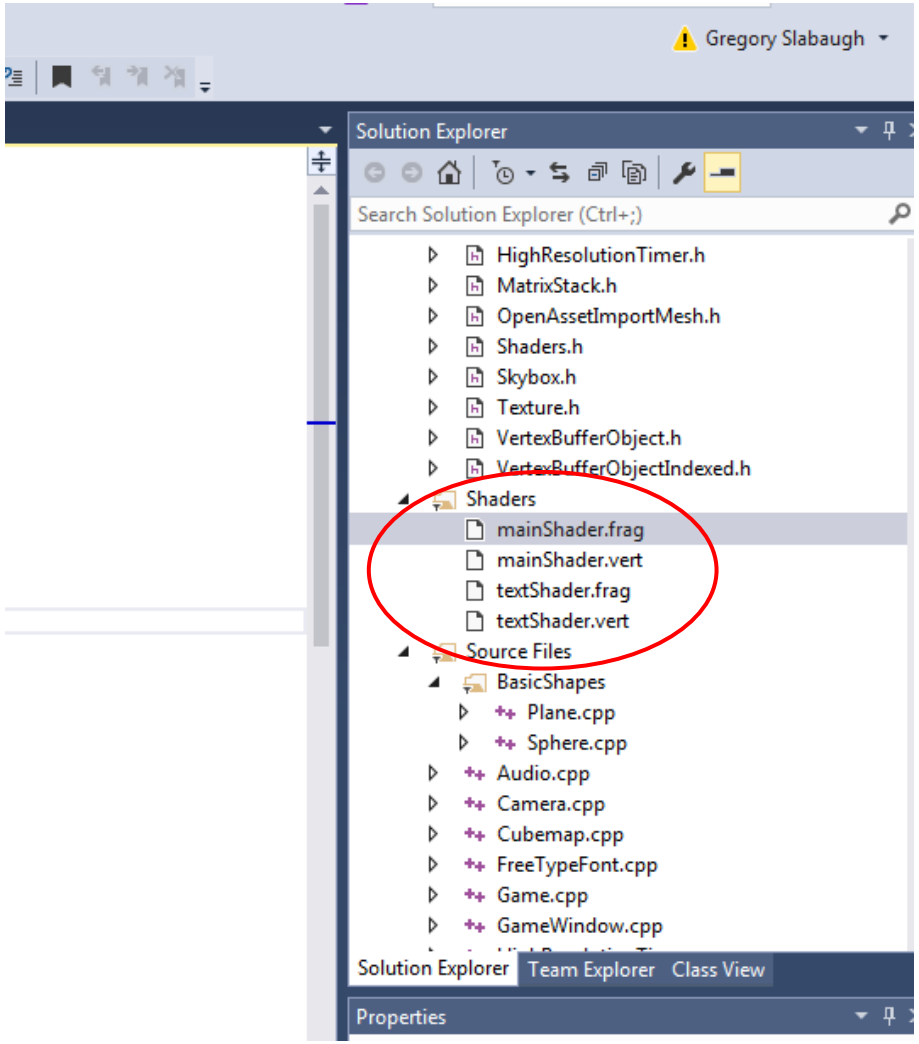


How does it work?

- The vertex and fragment shaders are each a C-like program that must be compiled, linked, and bound to the OpenGL rendering context.
- The client program provides the modelview and projection matrices to the shaders, and issues rendering commands, like `glDrawArrays` to render primitives.
- The vertex shader program, running on the graphics card, transforms the vertices. The vertex shader is responsible for at least writing a variable: **`gl_Position`**.
- The fragment shader program, also running on the graphics card, either discards the fragment (so it is not rendered) or assigns a colour to the fragment.

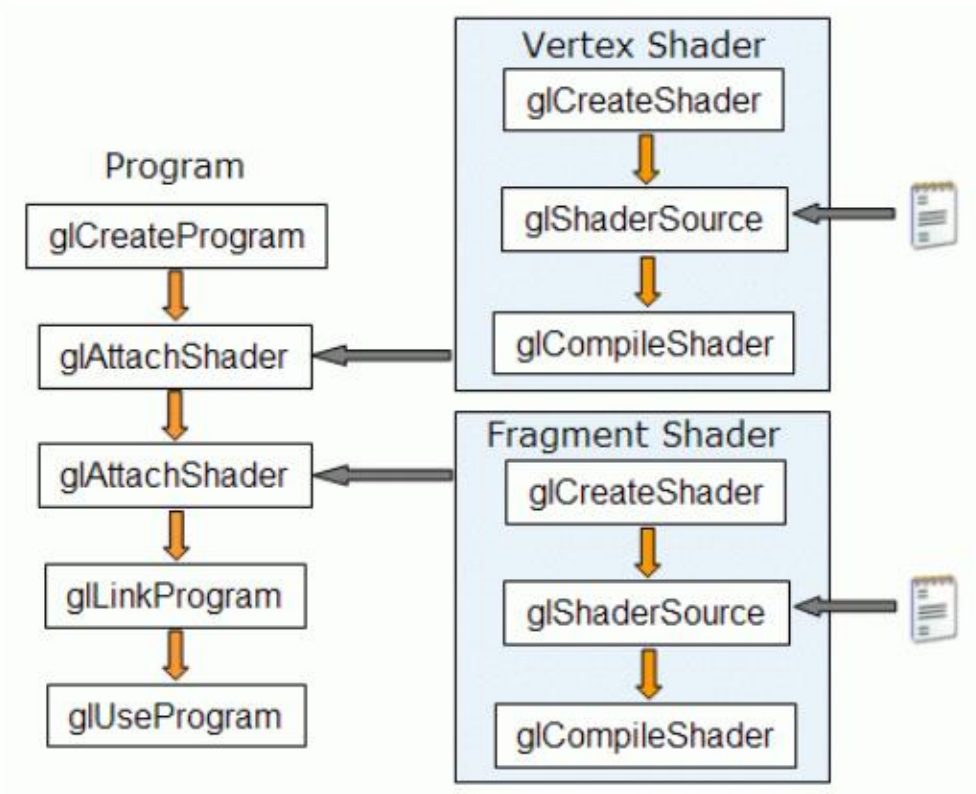
Shaders in the OpenGL Template

Code walkthrough



How does it work?

- A set of OpenGL functions are available to create, compile, attach, link, and use a shader. The compiler / linker is built into the graphics driver. Compilation and linking occur when invoked by the client programme.



Creating and compiling a shader

- For an individual shader, one first creates a shader ID:
  - `GLuint glCreateShader(GLenum shaderType);`
    - `shaderType` – `GL_VERTEX_SHADER`, `GL_FRAGMENT_SHADER`, `GL_GEOMETRY_SHADER`, (more)
    - This function returns a shader ID

- Shader code is often stored in a text file. The text file is then loaded into client memory, usually with a helper function. The next step provides the shader source code to the server:
  - `void glShaderSource(GLuint shaderID, int numOfStrings, const char **strings, int *lenOfStrings);`
    - `shaderID` – the shader ID
    - `numOfStrings` – the number of strings in the array
    - `strings` – the array of strings
    - `lenOfStrings` – an array with the length of each string, or NULL, meaning that the strings are NULL terminated

- Finally, the shader must be compiled:
  - `void glCompileShader(GLuint shaderID);`

Creating a program

- Next, we will create, link, and use a program that consists of both a vertex shader and a fragment shader.
- The first step is to create a program ID:
  - `GLuint glCreateProgram(void);`

- Once can create (and link) many programs. When rendering, one can switch from program to program.

Attaching the shaders

- The next step is to attach the shaders to the program. The shaders do not have to be compiled at this time or have source code. All that is required to attach a shader to a program is the shader ID.
- `void glAttachShader(GLuint programID, GLuint shaderID);`
  - `programID` – the ID of the program
  - `shaderID` – the ID of the shader to be attached
- If you have a pair of vertex/fragment shaders it is necessary to attach *both* to the program. A vertex/fragment shader pair is mandatory; geometry and tessellation shaders are optional.

Linking and using a program

- Next, one links the program. At this stage, the shaders must be compiled.
- void `glLinkProgram(GLuint programID);`
  - `programID` – the ID of the program

- Finally, to use the linked program, call
- void `glUseProgram(GLuint programID);`
  - `programID` – the ID of the program

⇨ A shader program will continue to run until another call to `glUseProgram` is made.

Example in the template code

The template code contains classes, CShader and CShaderProgram, which contain a collection of helper functions to load, compile, link, activate, and use shaders written in the OpenGL shading language.

```
// Load shaders
vector<CShader> shShaders;
vector<string> sShaderFileNames;
sShaderFileNames.push_back("mainShader.vert");
sShaderFileNames.push_back("mainShader.frag");
sShaderFileNames.push_back("textShader.vert");
sShaderFileNames.push_back("textShader.frag");

for (int i = 0; i < (int) sShaderFileNames.size(); i++) {
    string sExt = sShaderFileNames[i].substr((int) sShaderFileNames[i].size()-4, 4);
    int iShaderType;
    if (sExt == "vert") iShaderType = GL_VERTEX_SHADER;
    else if (sExt == "frag") iShaderType = GL_FRAGMENT_SHADER;
    else if (sExt == "geom") iShaderType = GL_GEOMETRY_SHADER;
    else if (sExt == "tcnl") iShaderType = GL_TESS_CONTROL_SHADER;
    else iShaderType = GL_TESS_EVALUATION_SHADER;
    CShader shader;
    shader.LoadShader("resources\\shaders\\"+sShaderFileNames[i], iShaderType);
    shShaders.push_back(shader);
}

// Create the main shader program
CShaderProgram *pMainProgram = new CShaderProgram;
pMainProgram->CreateProgram();
pMainProgram->AddShaderToProgram(&shShaders[0]);
pMainProgram->AddShaderToProgram(&shShaders[1]);
pMainProgram->LinkProgram();
m_pShaderPrograms->push_back(pMainProgram);
```

Errors

- Errors may be generated during compilation or linking.
- The status of the compilation or linking can be queried:
  `void glGetShaderiv(GLuint object, GLenum type, int *param);`
  where `object` is the `shaderID` or `programID`, `type` is `GL_COMPILE_STATUS` or `GL_LINK_STATUS`, and `param` is the return value: `GL_TRUE` if no error, otherwise, `GL_FALSE`
- For additional information, one must retrieve the *infolog*. This log stores warnings and errors in the compilation and problems during the link step.
  - First, get the size of the infolog:
    `glGetProgramiv(object, GL_INFO_LOG_LENGTH, &infologLength);`
  - Then, retrieve the log:
    `glGetProgramInfoLog(object, infologLength, &charsWritten, infoLog);`
- It's a good idea to check for errors after compiling and linking shaders.

⇨ On error, the template code pops up a message box.

Cleanup

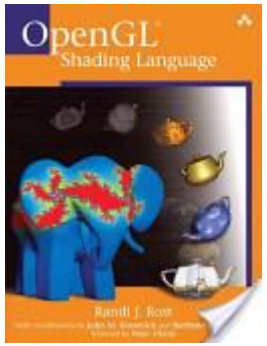
- A shader can be detached from a program using
```
void glDetachShader(GLuint programID, GLuint shaderID);
```
where `programID` is the program to detach from, and `shaderID` is the shader to detach.

- A shader can be deleted using
```
void glDeleteShader(GLuint shaderID);
```
  - Only unattached shaders can be deleted.

- A program can be deleted using
```
void glDeleteProgram(GLuint programID);
```

Shader demo

- “OpenGL Shading Language” by Randi Rost et al.
- It provides a explanation of how to program OpenGL shaders. Third edition covers OpenGL 3.1
- It has a compelling examples to highlight many of the capabilities of shader programming in OpenGL



Anatomy of a GLSL shader

- Each GLSL shader must have a `void main()` function. This is the entry point into the shader. Global variables may be declared above `main()`.
- The `main()` function may call other user-defined functions inside the shader. User-defined functions may return a value or be void.
- Simple example:

Fragment shader

```
#version 400

out vec4 fragColour;

void main()
{
     fragColour = vec4(0.4, 0.4, 0.8, 1.0);
}
```

Version of GLSL

Data types

- GLSL has five basic data types:
  - `float`
  - `int`
  - `uint`
  - `bool`
  - `sampler` (used for sampling textures)
- Vector types for `float`, `int`, `uint`, and `bool`:
  - `vec2`, `vec3`, `vec4`: 2D, 3D and 4D floating point vector
  - `ivec2`, `ivec3`, `ivec4`: 2D, 3D and 4D integer vector
  - `bvec2`, `bvec3`, `bvec4`: 2D, 3D and 4D boolean vector
- Matrix types for float:
  - `mat2`, `mat3`, `mat4`: 2x2, 3x3, 4x4 floating point matrix
- Sampler types:
  - `sampler1D`, `sampler2D`, `sampler3D`: for 1D, 2D, 3D textures
  - `samplerCube`: for cube map textures
  - others…
- Operator overloading for matrices, vectors as expected
- Vector and matrix types compatible with GLM!

Declaration and initialisation

- Declaring a variable is similar to C

```
vec3 f; // declaring f as a vec3
vec3 g = vec3(1.0,2.0,3.0); // declaring and initialising g
```

- Structures are allowed, e.g.

```
struct LightSource {
   vec3 direction;
   vec3 colour;
}
```

- Arrays are allowed:

```
float a[5] = float[5](3.4, 4.2, 5.0, 5.2, 1.1);
```

- GLSL is flexible when initializing variables using other variables

```
vec2 a = vec2(1.0, 2.0);
vec2 b = vec2(3.0, 4.0);
vec4 c = vec4(a, b) // c = vec4(1.0, 2.0, 3.0, 4.0);
vec3 d = vec3(1.0); // d = vec3(1.0, 1.0, 1.0);
mat4 m = mat4(1.0); // 4x4 identity matrix
vec2 e = vec2(1.0, 2.0);
vec2 f = vec2(3.0, 4.0);
mat2 n = mat2(e, f); // matrices are assigned in column major order
```

Vector and matrix element access

- Accessing a vector can be done using letters as well as standard C selectors.

```
vec4 a = vec4(1.0, 2.0, 3.0, 4.0);
float posX = a.x;
float posY = a[1];
vec2 posXY = a.xy;
float w = a.w;
```

- Letters (xyzw), (rgba), (stpq) used to access vector elements, for position, colour, texture coord respectively.
- **Swizzling** is an interesting syntax that allows multiple component access

```
vec2 a = vec2(1.0, 2.0);
vec4 b = a.xyxx;
vec3 c = b.zyy;
```

What is c?

- Matrix selectors can take one or two arguments
  - m[0] – the first column of the matrix
  - m[2][3] -- a single element in the matrix

Storage qualifiers

- Storage qualifiers are often used when communicating between the client and shaders, or between shaders.
  - (none)
  - `const`
  - `in`
  - `out`
- **const** variables are read only and used for declaring variables that don't change.
- **uniforms** are values which do not change during rendering, for example the light position or the light colour. *Uniforms are read-only and are passed from the client to the server*.
- **in** variables are coming into a shader, typically from an earlier shader stage. These variables may not be written to.
- **out** variables are pass from a shader, typically to a later shader stage.
- There are no pointers in GLSL.

Layout qualifier

- Vertex shaders allow input layout qualifiers on input variable declarations. The layout qualifier identifier for vertex shader inputs is:

```
layout (location = integer-constant)
```

- For example, to use the generic vertex attribute 0, representing vertex position, in the vertex shader:

```
layout (location = 0) in vec3 vPos;
```

This will establish that the vertex shader input `vPos` is copied in from location number 0. Recall the generic vertex attribute is configured when setting up the VBO, through the call `glVertexAttribPointer`.

Built-in variables

- For convenience, GLSL has some built-in variables for use in shaders. The importance of built-in variables is that they are mapped to the OpenGL states.
- Built-in variables for use in a vertex shader:
  - `in int gl_VertexID`: vertex index
  - `out vec4 gl_Position`: vector representing the vertex position
  - `out float gl_PointSize`: point rendering size
  - (others)
- Built-in variables for use in a fragment shader
  - `in vec4 gl_FragCoord`: Position of the fragment in window coordinates
  - `in bool gl_FrontFacing`: Flag indicating if fragment is front facing
  - `out float gl_FragDepth`: Depth value of the fragment
  - (others)

For a complete listing, see the GLSL 4.0 specification
http://www.opengl.org/registry/doc/GLSLangSpec.4.00.7.pdf

Hello world shader

Vertex shader

```
#version 400
uniform struct Matrices
{
   mat4 projMatrix;
   mat4 modelViewMatrix;
} matrices;

layout (location = 0) in vec3 inPosition;

void main()
{
   gl_Position = matrices.projMatrix * matrices.modelViewMatrix *
          vec4(inPosition, 1.0f);
}
```

Fragment shader

```
#version 400

out vec4 fragColour;

void main()
{
     fragColour = vec4(0.4, 0.4, 0.8, 1.0);
}
```

Control flow statements

- Control flow is very similar to C/C++:
  - `if/else`

```
if (bool expression)
...
else
...
```

- Loops
  - `for` loops

```
for (initialization; bool expression; loop expression)
...
```

  - `while` loops

```
while (bool expression)
...
```

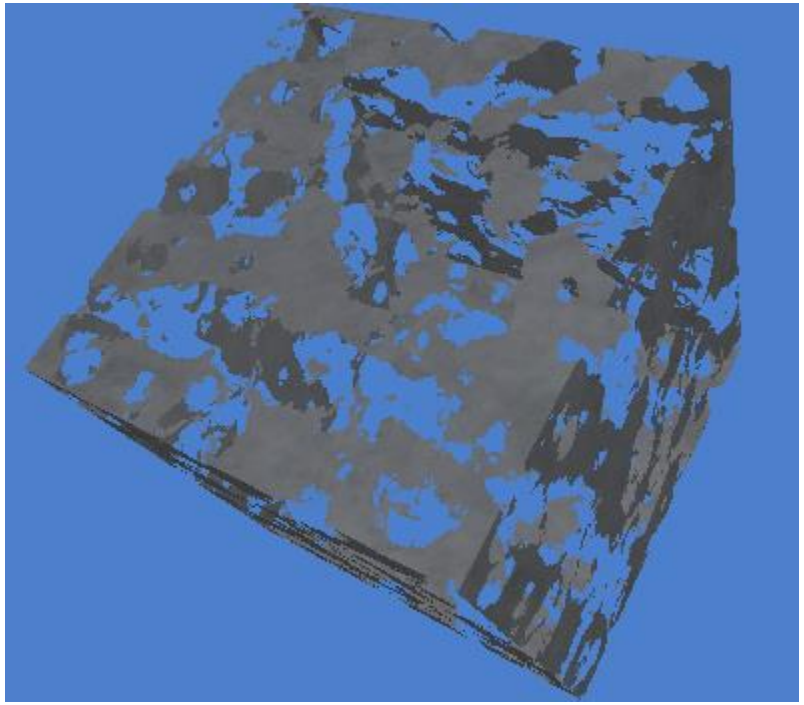
  - `do while` loops

```
do
...
while (bool expression)
```

  - `continue` – causes a jump to the next iteration of the loop
  - `break` – available in loops, causes an exit of the loop

Discard

- The `discard` command is *available only in a fragment shader*.
- It will cause the current fragment to not be rendered.
- Example: Using discard with a gray/brown texture, and discarding the brown fragments



User-defined functions

- Qualifiers exist for user-defined function parameters:
  - `in`: the parameter is an input variable
  - `out`: the parameter is an output variable
  - `inout`: the parameter is both an input and an output variable
- If no qualifier is used, by default it is considered to be `in`.
- A function can be overloaded as long as the list of parameters is different.
- Example of a user-defined function:

```
vec4 squareColour(in vec4 c) {
   vec4 colour = pow(c, vec4(2.0));
   return colour;
}
```

Built-in functions

- For convenience and efficiency, the OpenGL Shading Language defines an assortment of built-in functions for scalar and vector operations
- Built-in functions may
  - Expose some necessary hardware functionality in a convenient way.
  - Represent a common operations (clamp, mix, etc.) that are simple for the user to write, but may have direct hardware support.
  - Represent an operation graphics hardware is likely to accelerate at some point.

⇨ Programmers are encouraged to *use the built-in functions* rather than write equivalent code to perform similar computations.

Angle and trigonometry functions

radians	degrees	sin	asin
cos	acos	tan	atan
sinh	asinh	cosh	acosh
tanh	atanh		

In GLSL (and GLM), angles in trigonometric functions are expressed in *radians*

Exponential functions

pow	exp	log	exp2
log2	sqrt	inversesqrt	

Common functions

abs	sign	floor	ceil
fract	mod	min	max
clamp	mix	step	smoothstep
trunc	round	roundEven	modf
(more)			

- `clamp(x, minVal, maxVal)`: limits the value of x to a range [minVal, maxVal]
- `mix(x, y, a)`: linear interpolation, i.e. x·(1−a) + y·a
- `step(edge, x)`: returns 0.0 if x < edge, otherwise, 1.0
- `smoothstep(edge0, edge1, x)`: like step, but smoothly varying between edge0 and edge1

- What is z below?

```
float x = 4.0;
float y = 5.0;
float z =  mix( clamp(x, 0, 1), y, 0.5);
```

Answer: 3.0

Geometric functions

`length`	`distance`	`dot`	`cross`
`normalize`	`faceforward`	`reflect`	`refract`

- `faceforward(N, I, Nref)`: returns `N` if dot product of `I`, `Nref` is < 0; otherwise -N
- `reflect`, `refract`: used to bounce a vector off a surface or refract it through a surface

http://www.youtube.com/watch?v=fQVZ5Rni8X0

Matrix functions

matrixCompMult	outerProduct	transpose	determinant
inverse			

- `transpose(mat m)`: matrix transpose
- `inverse(mat m)`: returns the inverse matrix. Undefined if matrix is cannot be inverted (is *singular*).

Vector relational functions

`lessThan`	`lessThanEqual`	`equal`	`any`
`greaterThan`	`greaterThanEqual`	`notEqual`	`all`
`not`			

- Relational and equality operators (<, <=, >, >=, ==, !=) are overloaded and return a Boolean result for scalar arguments
- `lessThan`, `lessThanEqual`, `equal`, `greaterThan`, `greaterThanEqual` execute element-wise, and return a Boolean vector result
- `bool any(bvec x):` returns true if any component of x is true
- `bool all(bvec x):` returns true only if all components of x are true
- `bvec not(bvec x):` returns the element-wise logical complement of x

Texture lookup functions

`textureSize`	`texture`	`textureProj`	`textureLod`
`textureOffset`	`texelFetch`	`texelFetchOffset`	`textureProjOffset`
`textureLodOffset`	`textureProjLod`	`textureProjLodOffset`	`textureGrad`
`textureGradOffset`	`textureProjGrad`	`textureProjGradOffset`	

- These functions provide access to textures through samplers
- Texture properties defined using OpenGL API calls
- Typically one uses `texture`

Preprocessor

- Preprocessor directives

#	#define	#undef	#if
#ifdef	#ifndef	#else	#elif
#endif	#error	#pragma	#extension
#version	#line		

- #error: cause the implementation to put a diagnostic message into the info log
- #version: declare the version of GLSL they are written in. If not included, version 1.10 is assumed. The vertex and fragment shaders should have the same version. The #version directive should appear first if used. Example:

```
#version 400
```

- The version can be core as well

```
#version 400 core
```

Communication

- Often, one would like for the OpenGL client to pass values to shaders. This can done by getting the memory location of the variable in the shader after the shader has been linked. This can be achieved using

  ```
  GLint glGetUniformLocation(GLuint programID, const char *name);
  ```

  where `programID` is the program ID and `name` is the variable name. The return value is the location of the variable, which can then be used to assign values from the client to the shader.

⇨ Only communicate to shader programs that are activated, by calling `glUseProgram()` first.

Communication

- A uniform variable can be passed from the client to the shader using the location.
- There are a set of functions depending on whether the layout of the variable.
- Uniform (float below, replace “f” with “i” for integer):
  - `void glUniform1f(GLint location, GLfloat v0);`
  - `void glUniform2f(GLint location, GLfloat v0, GLfloat v1);`
  - `void glUniform3f(GLint location, GLfloat v0, GLfloat v1, GLfloat v2);`
  - `void glUniform4f(GLint location, GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3);`
  - `GLint glUniform{1,2,3,4}fv(GLint location, GLsizei count, GLfloat *v);`
  - `GLint glUniformMatrix{2,3,4}fv(GLint location, GLsizei count, GLboolean transpose, GLfloat *v);`

Communication in template

- The shader class in the template includes helper functions to communicate between the client program and the shaders.
- Function setUniform is overloaded depending on data type

```
void CShaderProgram::SetUniform(string sName, const float fValue)
{
   int iLoc = glGetUniformLocation(m_uiProgram, sName.c_str());
   glUniform1fv(iLoc, 1, &fValue);
}
```

```
pMainProgram->SetUniform("t", m_t);
```

What is shader doing?

Vertex shader

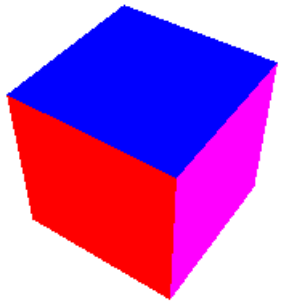
```
#version 400
layout (location = 0) in vec3 inPosition;

uniform struct Matrices
{
   mat4 projMatrix;
   mat4 modelViewMatrix;
} matrices;

void main()
{
   vec3 v = inPosition;
   v.z = 0.0;
   gl_Position = matrices.projMatrix *
          matrices.modelViewMatrix * vec4(v, 1.0f);
}
```

Fragment shader

```
#version 400
out vec4 fragColour;

void main()
{
     fragColour = vec4(0.8, 0.3, 0.3, 1);
}
```

Flattening a 3D surface to a plane

Colour shader



Vertex shader

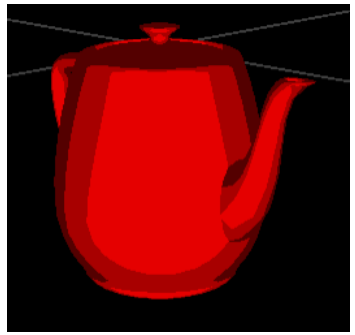
```
#version 400
layout(location = 0) in vec3 inPosition;
layout(location = 1) in vec3 inColour;
uniform struct Matrices
{
   mat4 projMatrix;
   mat4 modelViewMatrix;
} matrices;
out vec3 colour;
void main()
{
     gl_Position = matrices.projMatrix *
          matrices.modelViewMatrix * vec4(v, 1.0f);
     colour = inColour;
}
```

Fragment shader

```
#version 400
in vec3 colour;
out vec4 fragColour;

void main()
{
     fragColour = vec4(colour, 1.0);
}
```

Exercise: Basic toon shader

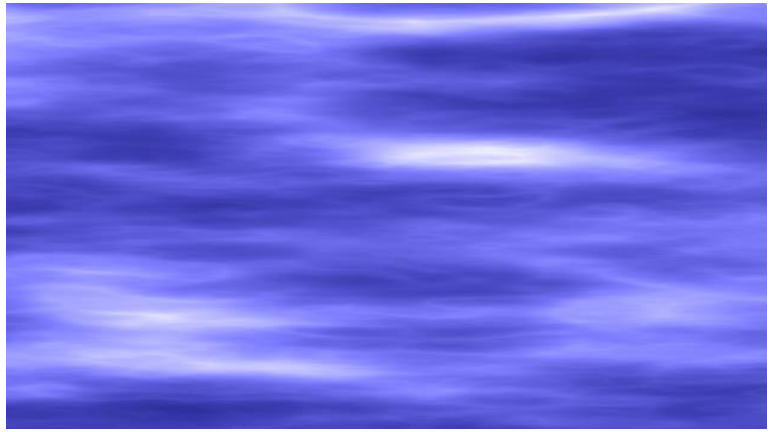
- A toon (or *cel*) shader creates a cartoon-like image. A common approach to toon shading is to limit the output colours to a set of fixed intensity levels, implemented in a fragment shader.
- Try writing your own toon shader to output three different levels of red based on the light intensity (`fIntensity`) at the fragment. For example,
  - if the intensity is greater than 0.75, output pure bright red (r = 1).
  - else if the intensity is greater than 0.5, output medium red (r = 0.6)
  - otherwise, output dark red (r = 0.3)
- Hint: Output colour (call it `fragColour`) is a vec4 – the last component is 1.0, i.e., (R, G, B, 1)



Fragment shader

Shadertoy

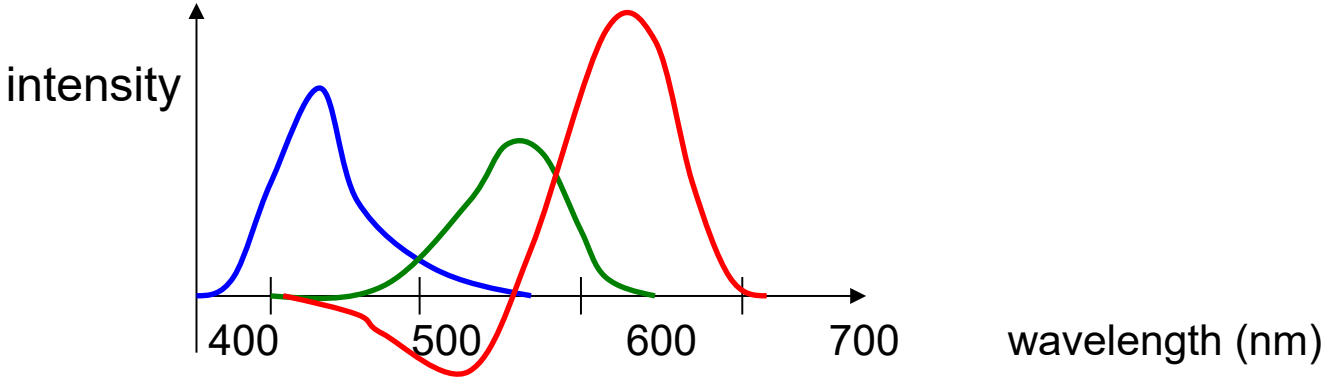
- Shadertoy is a website for demoing / sharing shaders written in WebGL.
- WebGL is a variant of OpenGL that runs in a web browser. The latest version is 1.0, which is based on OpenGL ES 2.0, which itself is based on OpenGL 2.0 without deprecated functionality. So WebGL is kind of like an older version of OpenGL.
- Shadertoy renders a fullscreen quad, and allows you to program a fragment shader.



https://www.shadertoy.com/view/llX3z2

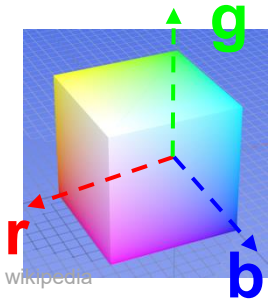
Colour Theory: Human Retina

- Tri-stimulus theory of vision:
  - Human eyes perceive colour through stimulation of three visual pigments in cones of retina.
  - One pigment has peak sensitivity to red, the other green, the other blue.
- Below shows the intensity of the combination of “pure” R, G and B light that can appear to a human like each wavelength/frequency of actual light.



- RGB colour: three colours are related to the physiology of the human eye

$$\mathbf{C} = \begin{bmatrix} r \\ g \\ b \end{bmatrix}$$



Source: wikipedia

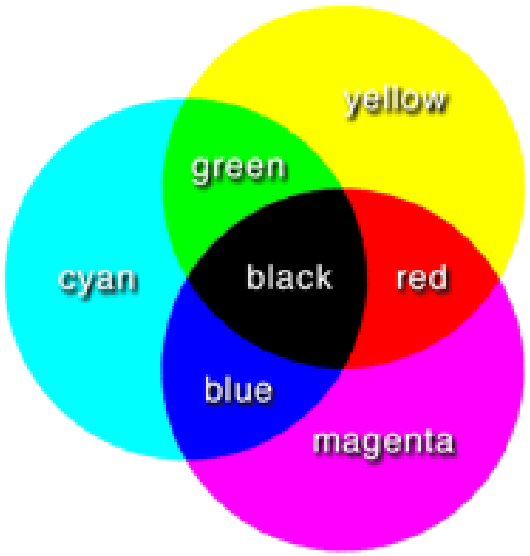
Colour

- Specification of colour is done as RGB (or RGBA) in a VBO
  - A colour `[0.1f, 0.9f, 0.9f]`^{T} specifies colour that emits 0.1 intensity red light, together with 0.9 intensity green light and 0.9 intensity blue light. Can you guess the colour?

- Grayscale refers to shades of grey, going from black to white. For grayscale, the red, green, and blue components are equal.

Other colour spaces: CMYK

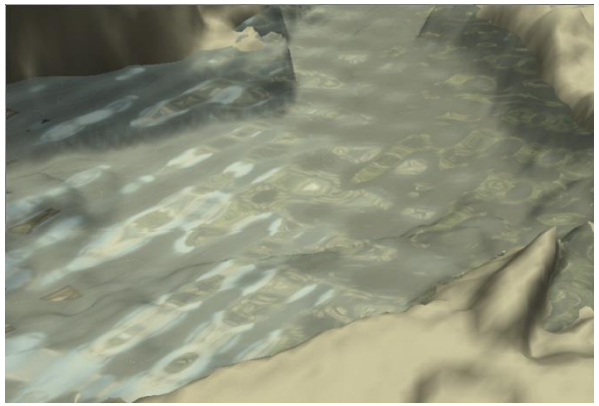
- Subtractive colour
- Used in printing
- Printers usually have Cyan, Magenta and Yellow ink, plus Black ink



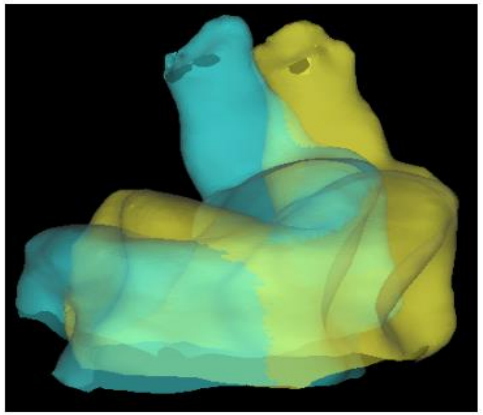
Alpha: the 4th colour component

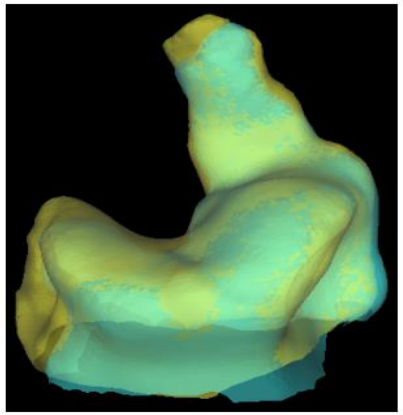
- The RGB model can be extended to incorporate transparency, using “alpha”. With transparency, it is possible to “see through” objects.
- In RGBA, the “A” stands for *alpha*, which measures **opacity** (note, this is the opposite of transparency)
  - Alpha = 1.0 : Fully opaque – cannot see through
  - Alpha = 0.0 : Fully transparent – can see through completely
- In RGB mode, alpha is assumed to be 1.0.

```
glEnable(GL_BLEND);
```



http://www.jayconrod.com

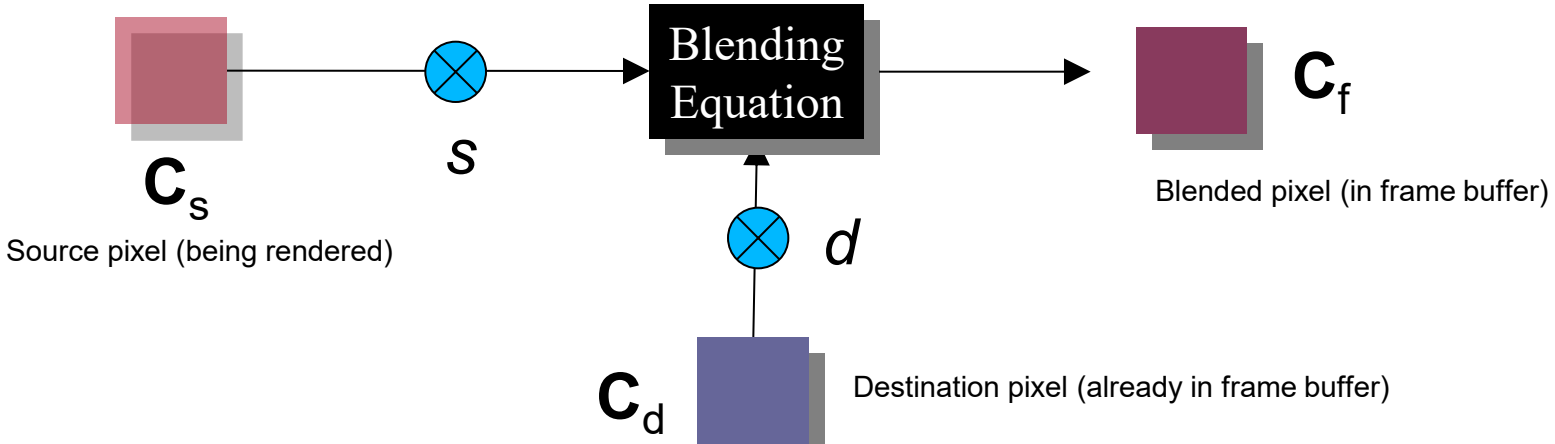




Alpha blending

- Alpha blending combines the colour of the current primitive with the colour already in the frame buffer.
- Terminology:
  - The colour value already stored in the frame buffer is the *destination colour* ($\mathbf{C}_d$)
  - The colour value that is coming into the frame buffer is the *source colour* ($\mathbf{C}_s$)
  - By default, the blending equation is

$$\mathbf{C}_f = s * \mathbf{C}_s + d * \mathbf{C}_d$$



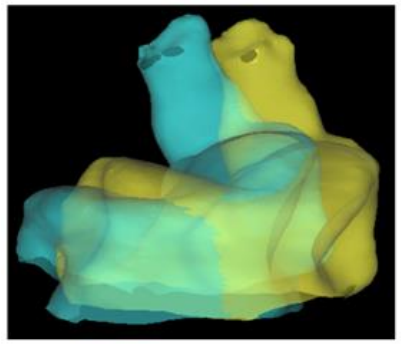
Alpha blending

- In the blending equation, *s* and *d* are blending factors, that describe how much of the source and destination colours to use when producing the blended result.
- In OpenGL, the blending factors are enumerants, not direct values. The blending factors are set with

```
glBlendFunc (GLenum s, GLenum d);
```

- There are many possible values *s* and *d* can take. Some of the common ones include
  - GL_ONE
  - GL_ZERO
  - GL_SRC_ALPHA
  - GL_ONE_MINUS_SRC_ALPHA

Alpha blending

- Perhaps the most common blending function is

  `glBlendFunc (GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);`
- This tells OpenGL to take the source colour and multiply it by the source alpha value. The destination colour is multiplied by one minus the source alpha. The result is the sum of these two terms.
- This blending function is often used to render a transparent object in front of an opaque object. To get the desired effect,
  - *Draw the opaque objects first before drawing the transparent ones*
  - If you're rendering back-facing polygons, it may be necessary to draw the transparent polygons far to near after depth sorting the polygons (based on distance to the camera). This is called the *painter's algorithm*.
  - Otherwise, use glEnable(GL_CULL_FACE) to eliminate back-facing triangles.